

实验二：LLM 工具调用扩展

实验二：LLM 工具调用扩展

实验目的

1. 掌握 LLM 工具调用的基本原理和实现方法
2. 学习如何设计和实现文件操作工具
3. 掌握网络访问工具的开发
4. 理解工具调用流程和响应解析机制

实验内容

实验过程

1. 代码结构分析 本实验包含两个 Python 文件：

chat_client.py - 文件操作工具客户端 - `list_files()`: 列出目录下的文件及其属性 - `format_size()`: 文件大小格式转换 - `rename_file()`: 重命名文件 - `delete_file()`: 删除文件 - `create_file()`: 创建新文件 - `read_file()`: 读取文件内容

tool_chat_client.py - 网络访问工具客户端 - `fetch_webpage()`: 通过 curl 获取网页内容 - `get_current_date()`: 获取当前日期时间

2. 工具调用机制 工具定义

```
TOOL_FUNCTIONS = {  
    'list_files': list_files,  
    'rename_file': rename_file,  
    'delete_file': delete_file,  
    'create_file': create_file,  
    'read_file': read_file  
}
```

工具调用流程 1. 用户输入消息 2. LLM 分析是否需要调用工具 3. LLM 输出 JSON 格式的工具调用指令 4. 解析工具调用并执行对应函数 5. 将工具执行结果返回给 LLM 6. LLM 生成最终总结回答

系统提示词设计

```
SYSTEM_PROMPT = """
```

你是一个文件操作助手，你可以执行以下文件操作：

可用工具：

1. `list_files(directory)` - 列出指定目录下的所有文件
2. `rename_file(directory, old_name, new_name)` - 重命名文件
3. `delete_file(directory, filename)` - 删除文件
4. `create_file(directory, filename, content)` - 创建文件
5. `read_file(directory, filename)` - 读取文件内容

格式要求：

- 当你需要调用工具时，请输出 JSON 格式：{"tool": "工具名称", "args": {"参数名": "参数值"}}
- ```
"""
```

## 3. 核心技术实现 工具调用解析

```
def parse_tool_call(response):
 try:
 response = response.strip()
 if response.startswith('{') and response.endswith('}'):
 data = json.loads(response)
 if 'tool' in data and 'args' in data:
 return {'tool': data['tool'], 'args': data['args']}
 return None
 except json.JSONDecodeError:
 return None
```

## 工具执行

```
def execute_tool(tool_name, args):
 if tool_name not in TOOL_FUNCTIONS:
 return {'success': False, 'error': f"未知工具: {tool_name}"}
 func = TOOL_FUNCTIONS[tool_name]
 try:
 return func(**args)
 except Exception as e:
 return {'success': False, 'error': str(e)}
```

## 实验结果

## 功能验证

| 功能     | chat_client.py | tool_chat_client.py |
|--------|----------------|---------------------|
| 列出目录文件 | 正确             | 错误                  |
| 文件重命名  | 正确             | 错误                  |
| 文件删除   | 正确             | 错误                  |
| 文件创建   | 正确             | 错误                  |
| 文件读取   | 正确             | 错误                  |
| 网页访问   | 错误             | 正确                  |
| 天气预报   | 错误             | 正确                  |
| 获取日期时间 | 错误             | 正确                  |

输出示例 运行 chat\_client.py 的典型输出：

```
=====
LLM 文件操作助手 - 工具调用演示
=====

API基础URL: https://api.example.com/v1
模型名称: gpt-3.5-turbo
温度参数: 0.7
最大令牌数: 1000
=====

支持的文件操作：
 1. 列出目录文件
 2. 重命名文件
 3. 删除文件
 4. 创建文件
 5. 读取文件
=====

请输入消息（输入 'exit' 或 'quit' 退出）：
> 列出当前目录的文件

正在发送请求...

=====

请求统计报告
=====

请求耗时: 0.5621 秒
输入令牌数 (Prompt): 45
输出令牌数 (Completion): 42
总令牌数: 87
=====
```

LLM响应

```
=====
{"tool": "list_files", "args": {"directory": "."}}
```

工具调用执行

调用工具: list\_files

参数: {"directory": "."}

执行结果: {

  "success": true,

  "data": [

    {"name": "chat\_client.py", "type": "file", "size": 15320, ...},

    {"name": "tool\_chat\_client.py", "type": "file", "size": 12450, ...}

  ]

}

正在获取总结回答...

总结回答

当前目录下有以下文件:

1. chat\_client.py - 文件操作工具客户端
2. tool\_chat\_client.py - 网络访问工具客户端

## 实验总结

### 实验成果

1. 成功实现了文件操作工具集，支持列出目录、创建、读取、重命名、删除文件
2. 实现了网络访问工具，支持网页获取和天气预报查询
3. 建立了完整的工具调用流程：解析 → 执行 → 总结
4. 实现了 JSON 格式的工具调用指令解析机制

### 技术要点

1. 工具注册机制: 通过字典映射工具名称和函数，便于扩展
2. 指令格式约定: 使用 JSON 格式传递工具调用指令，便于解析
3. 错误处理: 完善的错误处理机制，捕获网络异常和参数错误
4. 两阶段响应: 先调用工具获取数据，再生成总结回答

### 改进建议

1. 可添加工具调用权限控制，防止恶意操作
2. 可实现工具调用历史记录，便于审计
3. 可添加工具描述自动生成功能
4. 可支持多工具并行调用